

Poor queries:

1. master of software engineering
2. computer science
3. The
4. Course
5. Research
6. Faculty
7. Student
8. UCI
9. cristina lopes
10. the or and too but in a with is she my his you were are has

Good queries:

1. Acm
2. computer vision
3. intelligent systems
4. operating system
5. graduate student course
6. masters
7. cyber
8. cybersecurity
9. donald bren
10. Machine learning

The queries that were doing poorly initially were “master of software engineering,” “the,” and “the or and too but in a with is she my his you were are has” because of the presence of stopwords, as well as “UCI,” “course,” “research,” “faculty,” “student,” and “computer science” because of how common these terms were. Some specific words were ranked badly, like “cristina lopes”, due to their relevance in documents. We attempted to fix this issue by improving how we ranked the search query, as well as by improving the efficiency of the search engine.

To improve the effectiveness of the ranking, after we implemented the `tf_idf` ranking, we boosted the scores of the documents that contained the query terms in the headings, titles, or bold tags, since we realized we had not done that, and its absence was negatively impacting the way that results were appearing. This helped queries like “cristina lopes” get more relevant pages because pages like faculty profile pages tended to contain those terms in those tags. Besides fixing that mistake, we also realized that length normalization would help improve the ranking of the URLs because it helped shorter documents appear higher in the rankings rather than letting the longer documents dominate the top results. For queries that contained common terms, like “computer science,” which tended to be in longer documents, this was helpful because it allowed shorter documents that mentioned those terms fewer times, simply because of their length, to have an equal chance at being ranked.

While trying to improve the ranking, we were also able to make changes that improved the runtime performance of the search. One of the queries that was performing a little worse was “the or and too but in a with is she my his you were are has,” likely because of the length of the query, so we took some steps to improve the search speed. We did this by looking for places where the data structure being used was not the most efficient one, such as by converting a list to a set to improve the time complexity to $O(1)$ from $O(n)$ on lookup. One such instance of this was in our function that implemented tfidf ranking, where the search result that was passed in was stored in a set instead of a list, so that lookup for a docID was $O(1)$ instead of $O(n)$ if it was not in the search result. Another way we improved the runtime performance was by ensuring that code that only needed to run once only ran once to avoid using unnecessary resources. One place we did this was with the length of the documents, which we stored for later use with length normalization in our tfidf implementation, so we only calculated the document lengths once and kept using that.